

Manual Técnico YFERA Framework

Desarrollado por: Javier Alejandro Mérida Gómez - 202230638

Descripción general

YFERA es un framework personalizado para la construcción de aplicaciones web. Está diseñado para ser fácil de aprender, tipado y potente en la generación de múltiples vistas. El framework incluye un entorno de desarrollo integrado (IDE) que permite a los usuarios crear, editar y publicar sus propias aplicaciones web en un entorno de producción.

- El IDE incorpora características avanzadas como:
- Exportación del árbol de trabajo.
- Creación y manipulación de archivos y carpetas.
- Apertura de proyectos.
- Reconocimiento de comentarios en el código.
- Notificaciones claras y amigables de errores léxicos o sintácticos.
- Selector de colores personalizado.
- Resaltado de sintaxis y auto indentación.

Objetivos

- Proveer un framework ligero y tipado para la generación de vistas web.
- Implementar un IDE funcional que facilite el desarrollo de aplicaciones usando YFERA.
- Soportar la publicación rápida de aplicaciones a un entorno de producción.

Especificaciones del Sistema

Requisitos de Hardware y Software

- Sistemas Operativos Compatibles: Microsoft Windows 10 (o superior) / GNU/Linux (Ubuntu 20.04 LTS o superior).
- Entorno de Desarrollo (Core): Visual Studio Code.
- Navegador Web: Motores basados en Chromium (Chrome, Edge) o Firefox (versiones actualizadas).

Tecnologías y Librerías

Desarrollo	Tecnología / Librerías	Función
Analizadores	JISON	Generar los analizadores léxicos y sintácticos por lenguaje
Frontend	Angular	Interfaz
Backend	JavaScript (Node.js)	Lógica
Librerías fronted	lucide-angular, monaco-editor jszip file-saver @types/file-saver --save-dev	Editor API Exportar archivos
Librerías backend	express, cors	Conexión frontend-backend
Base de Datos	SQLite	Almacenamiento

Repositorio en GitHub

Link del repositorio:

https://github.com/JavierMeridaLk/ProyectoDosCompiladores1_2026

Arquitectura del Software

La potencia de YFERA radica en la integración de cuatro sub-lenguajes especializados. A continuación, se detallan las gramáticas completas y sus definiciones léxicas.

Lenguaje de Estilos (.styles)

Encargado de la definición estética de los componentes.

Tokens

A continuación se definen los elementos léxicos reconocidos por el analizador de estilos.

Identificadores y Valores:

IDENTIFICADOR

NUMERO (Deriva en DECIMAL o ENTERO)

VARIABLE

Palabras Reservadas (Propiedades y Atributos):

Dimensiones: height, width, min-width, max-width, min-height, max-height

Color y Fondo: background color, color

Texto: text align, text size, text font, font

Espaciado (Padding): padding, padding left, padding top, padding right, padding bottom

Márgenes (Margin): margin, margin left, margin top, margin right, margin bottom

Bordes: border radius, border style, border width, border color, border, border top style, border right

Directivas y Ciclos: extends, @for, through, from, to

Unidades: px

Valores Específicos Constantes:

Dirección: CENTER, RIGHT, LEFT

Fuentes: HELVETICA, SANS, SANS SERIF, MONO, CURSIVE

Estilos de Borde: DOTTED, LINE, DOUBLE

Símbolos y Operadores:

Agrupación y Asignación: {, }, =, :, \$

Aritméticos: +, -, *, /, %, - (unitario)

Relacionales y Lógicos: >, >=, <, <=, ==, !=, ||, &&, !

Gramática:

A continuación se definen los elementos sintácticos reconocidos por el analizador de estilos.

```
estilo := IDENTIFICADOR LLAVE_IZQ lista_atributos LLAVE_DER
        | IDENTIFICADOR PR_EXTENDS IDENTIFICADOR LLAVE_IZQ
        lista_atributos LLAVE_DER
```

```
lista_atributos := propiedades
                | formato_texto
                | espacios_internos
```

- | espacios_externos
- | bordes
- | lista_atributos

propiedades := PR_HEIGHT SIGNO_IGUAL numero PUNTO_COMA
| PR_WIDTH SIGNO_IGUAL numero PUNTO_COMA
| PR_MIN_WIDTH SIGNO_IGUAL numero PUNTO_COMA
| PR_MAX_WIDTH SIGNO_IGUAL numero PUNTO_COMA
| PR_MIN_HEIGHT SIGNO_IGUAL numero PUNTO_COMA
| PR_MAX_HEIGHT SIGNO_IGUAL numero PUNTO_COMA
| PR_BACKGROUND_COLOR SIGNO_IGUAL color PUNTO_COMA
| PR_COLOR SIGNO_IGUAL colo PUNTO_COMA

formato_texto := PR_TEXT_ALIGN SIGNO_IGUAL direccion PUNTO_COMA
| PR_TEXT_SIZE SIGNO_IGUAL numero PUNTO_COMA
| PR_TEXT_FONT SIGNO_IGUAL font PUNTO_COMA

font:= HELVETICA
| SANS
| SANS SERIF
| MONO
| CURSIVE

direccion := CENTER
| RIGHT
| LEFT

numero := ENTERO
| DECIMAL

espacios_internos := PR_PADDING SIGNO_IGUAL numero PUNTO_COMA
| PR_PADDING_LEFT SIGNO_IGUAL numero
PUNTO_COMA
| PR_PADDING_TOP SIGNO_IGUAL numero
PUNTO_COMA
| PR_PADDING_RIGHT SIGNO_IGUAL numero
PUNTO_COMA
| PR_PADDING_BOTTOM SIGNO_IGUAL numero
PUNTO_COMA

espacios_externos := PR_MARGIN SIGNO_IGUAL numero PUNTO_COMA
| PR_MARGIN_LEFT SIGNO_IGUAL numero
PUNTO_COMA

```

| PR_MARGIN_TOP SIGNO_IGUAL numero
PUNTO_COMA
| PR_MARGIN_RIGHT SIGNO_IGUAL numero
PUNTO_COMA
| PR_MARGIN_BOTTOM SIGNO_IGUAL numero
PUNTO_COMA

bordes := PR_BORDER_RADIUS SIGNO_IGUAL numero PUNTO_COMA
| PR_BORDER_STYLE SIGNO_IGUAL style_border
PUNTO_COMA
| PR_BORDER_WIDTH SIGNO_IGUAL numero
PUNTO_COMA
| PR_BORDER_COLOR SIGNO_IGUAL color PUNTO_COMA
| PR_BORDER SIGNO_IGUAL (ALGO) PUNTO_COMA
| PR_BORDER_TOP_STYLE SIGNO_IGUAL (ALGO)
PUNTO_COMA
| PR_BORDER_RIGHT SIGNO_IGUAL (ALGO) PUNTO_COMA

style_border := DOTTED
| LINE
| DOUBLE

operador_aritmetico := SUMA
| RESTA
| MULTIPLICACION
| DIVISION
| PORCENTAJE
| PAR_DER
| PAR_IZQ
| UNITARIO

```

Lenguaje de Componentes (.comps)

Define la estructura jerárquica y los elementos visuales de la aplicación.}

Tokens

Definición de los elementos léxicos para el estructurado de componentes visuales y formularios.

Identificadores y Tipos:

VARIABLES | IDENTIFICADORES

int, string, true, false

Palabras Reservadas:

Estructura: component, function

Elementos Visuales: T, IMG, FROM, SUBMIT

Inputs: INPUT_TEXT, INPUT_NUMBER, INPUT_BOOL

Atributos de Input: id, label, value

Estructuras de Control: @, for, each, track, empty, if, else, Switch, case, default

Símbolos y Operadores:

Agrupación: {, }, (,), [,], [[,]]

Puntuación: <, >, ", ,, :

Aritméticos: +, -, *, /, %, - (unitario)

Relacionales y Lógicos: >, >=, <, <=, ==, !=, ||, &&, !}

Gramática

componente := PR_COMPONENT PAR_IZQ PAR_DER LLAVE_IZQ elemento
LLAVE_DER
 | PR_COMPONENT PAR_IZQ atributos PAR_DER LLAVE_IZQ
elemento LLAVE_DER

atributo := type IDENTIFICADOR
 | type IDENTIFICADOR COMA atributo

elemento := secciones
 | tablas
 | texto
 | imagenes
 | formularios

secciones := PR_COR_IZQ_SIMPLE elemento PR_COR_DER_SIMPLE
 | estilo PR_COR_IZQ_SIMPLE elemento PR_COR_DER_SIMPLE

tablas := PR_COR_IZQ_DOBLE lista_filas PR_COR_DER_DOBLE
 | estilo PR_COR_IZQ_DOBLE lista_filas PR_COR_DER_DOBLE

lista_filas := lista_filas fila
 | fila

fila := PR_COR_IZQ_DOBLE lista_columnas PR_COR_DER_DOBLE

lista_columnas := lista_columnas columna
| columna

columna := PR_COR_IZQ_DOBLE elemento PR_COR_DER_DOBLE

texto := PR_T PAR_IZQ COMILLAS TEXTO COMILLAS PAR_DER
| PR_T estilo PAR_IZQ COMILLAS TEXTO COMILLAS PAR_DER

imagenes := PR_IMG lista_img
| PR_IMG estilo lista_img

lista_img := texto COMA lista_img
| PR_\$ IDENTIFICADOR COMA lista_img
| texto
| PR_\$ IDENTIFICADOR

formularios := LLAVE_IZQ PR_FORM LLAVE_IZQ lista_formulario LLAVE_DER
submit LLAVE_DER
| LLAVE_IZQ PR_FORM estilo LLAVE_IZQ lista_formulario
LLAVE_DER submit LLAVE_DER
| LLAVE_IZQ PR_FORM LLAVE_IZQ lista_formulario LLAVE_DER
LLAVE_DER
| LLAVE_IZQ PR_FORM estilo LLAVE_IZQ lista_formulario
LLAVE_DER LLAVE_DER

submit := PR_SUBMIT estilo LLAVE_IZQ propiedades_submit LLAVE_DER

propiedades_submit := PR_LABEL DOS_PUNTOS COMILLAS TEXTO COMILLAS

lista_formulario := elementos lista_formulario
| lista_input lista_formulario
| lista_input
| elementos

lista_input := input_text
| input_number
| input_bool

input_text := PR_INPUT_TEXT estilo LLAVE_IZQ propiedades_form LLAVE_DER
| PR_INPUT_TEXT LLAVE_IZQ propiedades_form LLAVE_DER

input_number := PR_INPUT_NUMBER estilo LLAVE_IZQ propiedades_form
LLAVE_DER
| PR_INPUT_NUMBER LLAVE_IZQ propiedades_form LLAVE_DER

input_bool := PR_INPUT_BOOL estilo LLAVE_IZQ propiedades_form LLAVE_DER
| PR_INPUT_BOOL LLAVE_IZQ propiedades_form LLAVE_DER

propiedades_form := PR_ID DOS_PUNTOS valor COMA PR_LABEL DOS PUNTOS
valor COMA PR_VALUE DOS PUNTOS valor

valor := COMILLAS TEXTO COMILLAS
| COMILLAS VAR COMILLAS
| VAR
| PR_TRUE
| PR_FALSE

Lenguaje de Base de Datos

Lenguaje declarativo para la gestión de persistencia de datos.

Tokens

Elementos léxicos destinados a la creación, inserción y consulta de datos.

Palabras Reservadas e Identificadores:

TABLE, COLUMNS, IN, DELETE

IDENTIFICADOR, NUMERO

TYPE (Nota: Utiliza los mismos tipos de datos definidos en el Lenguaje Principal).

Símbolos y Operadores:

Puntuación: =, ,, :, ., [,], "

Aritméticos: +, -, *, /, %, - (unitario)

Relacionales y Lógicos: >, >=, <, <=, ==, !=, ||, &&, !

consulta ::= query ";"

crear_tabla ::= "TABLE" IDENTIFICADOR "COLUMNS" columnas

columnas ::= IDENTIFICADOR "=" type ","

| IDENTIFICADOR "=" type

selec ::= IDENTIFICADOR "." IDENTIFICADOR

crear_regis ::= IDENTIFICADOR "[" "]"

Lenguaje Principal (.y)

Lenguaje declarativo para la gestión de persistencia de datos.

Tokens

El motor principal del sistema, que orquesta la ejecución lógica, variables y flujos de control.

Tipos de Datos y Literales:

int, float, string, boolean, char

IDENTIFICADOR, NUMERO, TEXTO

True, False

Palabras Reservadas (Instrucciones y Control):

Estructura y Funciones: import, execute, function, load, main

Decoradores/Macros: @header(), @lineForStyle, @Card, @cardForProfOak, @cardForAsh, @footer

Ciclos y Condicionales: while, for, do, if, else, switch, case, break, default

Símbolos y Operadores:

Agrupación y Puntuación: ", ' , ; , [,] , { , } , , , (,) , =

Aritméticos: +, -, *, /, %, - (unitario)

Relacionales y Lógicos: >, >=, <, <=, ==, !=, ||, &&, !

Gramática

codigo := imports definicion_variables PR_MAIN LLAVE_IZQ contenido_main
LLAVE_DER bloque_codigo

imports :=

definicion_variables := linea_variable

linea_variable := type COR_IZQ COR_DER IDENTIFICADOR SIGNO_IGUAL
COR_IZQ NUMERO COR_DER PUNTO_COMA linea_variable
| type IDENTIFICADOR SIGNO_IGUAL valor PUNTO_COMA
linea_variable
| type IDENTIFICADOR SIGNO_IGUAL valor PUNTO_COMA

bloque_codigo := bloque_if
| bloque_switch
| bloque_ciclos

bloque_if := PR_IF condicion LLAVE_IZQ contenido_if LLAVE_DER else_if
| PR_IF condicion LLAVE_IZQ contenido_if LLAVE_DER else
| PR_IF condicion LLAVE_IZQ contenido_if LLAVE_DER

else := PR_ELSE LLAVE_IZQ contenido LLAVE_DER

else_if := PR_ELSE PR_IF LLAVE_IZQ contenido LLAVE_DER

contenido_if := bloque_codigo

condicion := PAR_IZQ NUMERO SIMBOLO_LOGICO NUMERO PAR_DER

bloque_switch := PR_SWITCH condicion_switch LLAVE_IZQ contenido_switch
LLAVE_DER

condicion_switch :=

contenido_switch := case contenido_switch
| case PR_DEFAULT DOS_PUNTOS PR_BRAKE

case := PR_CASE COMILLAS TEXTO COMILLAS DOS_PUNTOS accion_switch
| PR_CASE COMILLAS TEXTO COMILLAS DOS_PUNTOS accion_switch
PR_SWITCH

while := PR_WHILE condicion LLAVE_IZQ contenido_while LLAVE_DER

do_while := PR_DO LLAVE_IZQ contenido_do LLAVE_DER condicion

for := PR_FOR condicion_for LLAVE_IZQ contenido_for LLAVE_DER

condicion_for := PAR_IZQ variable_temporal SIGNO_IGUAL PUNTO_COMA
variable_temporal SIGNO_MENOS SIGNO_IGUAL NUMERO PUNTO_COMA
variable_temporal SIGNO_IGUAL variable signo_aritmetico NUMERO PAR_DER

signos_aritmeticos := SUMA	(precedencia de 1)
RESTA	(precedencia de 1)
MULTIPLICACION	(precedencia de 2)
DIVISION	(precedencia de 2)
MODULO	(precedencia de 3)
PARÉNTESIS	(precedencia de 4)
UNITARIO	(precedencia de 4)

operadores_comparasion := MAYOR_QUE	(precedencia de 0)
MAYOR_IGUAL	(precedencia de 0)
MENOR_QUE	(precedencia de 0)
MENOR_IGUAL	(precedencia de 0)
IGUALDAD	(precedencia de 0)
DIFERENTE	(precedencia de 0)

operadores_logicos := OR	(precedencia de 1)
AND	(precedencia de 1)
NOT	(precedencia de 2)

Implementación del IDE

Editor y resaltado de sintaxis

Se utiliza Monaco Editor (mismo núcleo que VS Code) para proporcionar resaltado de sintaxis, auto indentación.

Los analizadores JISON se integran al backend; cada vez que el usuario escribe, se envía el texto a analizar y se devuelven los errores en tiempo real.

Gestión de proyectos

El IDE permite crear, abrir, guardar y exportar proyectos.

La estructura de archivos se muestra en un árbol lateral.

Se soporta creación de carpetas y archivos con extensiones .y(YFERA), .styles(estilos) y .comp(componentes).

Notificación de errores

Los errores léxicos y sintácticos se muestran en un apartado especial para errores con mensajes amigables, indicando línea, columna y una sugerencia de corrección.

Diagrama de clases